

MATERI PEMBELAJARAN

Function (Fungsi)

Deklarasi, Parameter, Return Value & Rekursi dalam C++

Pemrograman C++ | Tingkat Menengah

Fungsi (*function*) adalah blok kode yang dirancang untuk melakukan tugas tertentu dan dapat dipanggil berkali-kali dari bagian manapun dalam program. Fungsi adalah fondasi utama pemrograman terstruktur — membuat kode lebih modular, mudah dibaca, dan dapat digunakan ulang (*reusable*).

Peta Materi:

No.	Topik	Deskripsi Singkat
1	Anatomi Fungsi	Struktur dasar: deklarasi, definisi, pemanggilan
2	Tipe Return Value	void, nilai tunggal, multiple (pair/tuple)
3	Parameter Fungsi	By value, by reference, by pointer, default
4	Function Overloading	Banyak fungsi satu nama, parameter berbeda
5	Fungsi Rekursif	Fungsi yang memanggil dirinya sendiri
6	Inline Function	Optimasi pemanggilan fungsi kecil
7	Lambda Function	Fungsi anonim (C++11)

No.	Topik	Deskripsi Singkat
8	Function Pointer	Pointer yang menunjuk ke fungsi
9	Scope & Storage Class	Local, global, static, extern
10	Template Function	Fungsi generik untuk berbagai tipe data

1. Anatomi Fungsi

Sebuah fungsi di C++ terdiri dari empat bagian utama: **tipe return**, **nama fungsi**, **daftar parameter**, dan **body**.

Struktur Umum:

```
tipe_return namaFungsi(tipe1 param1, tipe2 param2) {
// body fungsi
return nilai; // jika tipe_return bukan void
}
```

Tiga Tahap Penggunaan Fungsi:

```
#include <iostream>
using namespace std;

// 1. DEKLARASI (prototype) – beritahu compiler sebelum main()
int luasSegitiga(int alas, int tinggi);

// 2. DEFINISI – implementasi detail fungsi
int luasSegitiga(int alas, int tinggi) {
return (alas * tinggi) / 2;
}

int main() {
// 3. PEMANGGILAN
int luas = luasSegitiga(10, 6);
cout << "Luas: " << luas << endl; // 30
return 0;
}
```

■ Deklarasi (prototype) wajib ada sebelum pemanggilan jika definisi fungsi ditulis setelah main(). Jika definisi ditulis sebelum main(), prototype tidak diperlukan.

2. Tipe Return Value

Fungsi dapat mengembalikan satu nilai menggunakan **return**, atau tidak mengembalikan nilai sama sekali (**void**). C++ juga mendukung pengembalian beberapa nilai menggunakan pair atau tuple.

a) void — Tidak Mengembalikan Nilai:

```
void cetakGaris(int panjang) {  
    for (int i = 0; i < panjang; i++) cout << '-';  
    cout << endl;  
    // tidak ada return (atau return; saja)  
}
```

b) Return Nilai Tunggal:

```
double luasLingkaran(double r) {  
    const double PI = 3.14159265;  
    return PI * r * r;  
}  
  
bool adalahGenap(int n) {  
    return (n % 2 == 0);  
}  
  
string sapaUser(string nama) {  
    return "Halo, " + nama + "!";  
}
```

c) Return Beberapa Nilai (pair & tuple, C++11):

```

#include <iostream>
#include <tuple>
using namespace std;

// Mengembalikan 2 nilai dengan pair
pair<int, int> minMax(int arr[], int n) {
    int mn = arr[0], mx = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < mn) mn = arr[i];
        if (arr[i] > mx) mx = arr[i];
    }
    return {mn, mx};
}

// Mengembalikan 3 nilai dengan tuple
tuple<double,double,double> statistik(double a, double b, double c) {
    return {a+b+c, (a+b+c)/3, a*b*c};
}

int main() {
    int data[] = {5, 2, 8, 1, 9, 3};
    auto [mn, mx] = minMax(data, 6); // C++17 structured binding
    cout << "Min: " << mn << ", Max: " << mx << endl;

    auto [total, rata, kali] = statistik(2.0, 4.0, 6.0);
    cout << "Total: " << total << ", Rata: " << rata << endl;
    return 0;
}

```

3. Parameter Fungsi

C++ mendukung beberapa cara pengiriman argumen ke fungsi. Pilihan yang tepat sangat mempengaruhi performa dan perilaku program.

Cara Passing	Sintaks	Nilai Asli Berubah?	Kapan Digunakan?
By Value	int f(int x)	Tidak	Data kecil, tidak perlu ubah nilai asli
By Reference	int f(int& x)	Ya	Ubah nilai asli / hindari copy data besar
By Const Ref	int f(const int& x)	Tidak	Data besar, hanya baca (efisien)
By Pointer	int f(int* x)	Ya (via *x)	Opsional null, array, low-level

Contoh Perbandingan:

```
#include <iostream>
using namespace std;

void byValue(int x) { x = 999; } // tidak berpengaruh ke luar
void byReference(int& x) { x = 999; } // ubah nilai asli
void byPointer(int* x) { *x = 999; } // ubah nilai asli
void byConstRef(const int& x) { // hanya baca, tidak bisa ubah
// x = 100; // ERROR: const!
cout << "Baca saja: " << x << endl;
}

int main() {
int a = 10, b = 10, c = 10;
byValue(a); cout << "byValue : " << a << endl; // 10 (tidak berubah)
byReference(b); cout << "byReference: " << b << endl; // 999
byPointer(&c); cout << "byPointer : " << c << endl; // 999
byConstRef(a);
return 0;
}
```

Default Parameter:

```
// Parameter dengan nilai default (harus di kanan)
void cetakInfo(string nama, int umur = 0, string kota = "Unknown") {
cout << nama << " | " << umur << " | " << kota << endl;
}

int main() {
cetakInfo("Budi", 20, "Makassar"); // Budi | 20 | Makassar
cetakInfo("Siti", 25); // Siti | 25 | Unknown
cetakInfo("Andi"); // Andi | 0 | Unknown
return 0;
}
```

■ Default parameter harus diletakkan di bagian paling kanan daftar parameter. Tidak bisa meletakkan default parameter di tengah jika parameter kanan tidak punya default.

4. Function Overloading

Function Overloading memungkinkan kita mendefinisikan beberapa fungsi dengan **nama yang sama** tetapi **parameter yang berbeda** (jumlah atau tipenya). Compiler akan memilih fungsi yang tepat berdasarkan argumen yang diberikan.

```
#include <iostream>
using namespace std;

// Satu nama 'luas', tiga versi berbeda
double luas(double sisi) { // persegi
return sisi * sisi;
}
double luas(double p, double l) { // persegi panjang
return p * l;
}
double luas(double alas, double tinggi, bool segitiga) { // segitiga
return 0.5 * alas * tinggi;
}

// Overloading berdasarkan tipe parameter
void cetak(int x) { cout << "int: " << x << endl; }
void cetak(double x) { cout << "double: " << x << endl; }
void cetak(string x) { cout << "string: " << x << endl; }

int main() {
cout << luas(5.0) << endl; // 25
cout << luas(4.0, 6.0) << endl; // 24
cout << luas(3.0, 8.0, true) << endl; // 12

cetak(42);
cetak(3.14);
cetak("Halo!");
return 0;
}
```

■ *Overloading ditentukan oleh jumlah dan tipe parameter, BUKAN oleh tipe return value. Dua fungsi dengan nama dan parameter sama tapi return berbeda akan menyebabkan error.*

5. Fungsi Rekursif

Rekursi adalah teknik di mana sebuah fungsi **memanggil dirinya sendiri**. Setiap fungsi rekursif wajib memiliki **base case** (kondisi berhenti) untuk mencegah rekursi tak terbatas.

Komponen	Penjelasan	Contoh
Base Case	Kondisi berhenti rekursi	if (n == 0) return 1;
Recursive Case	Memanggil dirinya dengan input lebih kecil	return n * faktorial(n-1);

Contoh 1 — Faktorial:

```

#include <iostream>
using namespace std;

long long faktorial(int n) {
    if (n == 0 || n == 1) return 1; // base case
    return n * faktorial(n - 1); // recursive case
}

// Trace faktorial(4):
// faktorial(4) = 4 * faktorial(3)
// = 4 * 3 * faktorial(2)
// = 4 * 3 * 2 * faktorial(1)
// = 4 * 3 * 2 * 1 = 24

int main() {
    for (int i = 0; i <= 10; i++)
        cout << i << "! = " << faktorial(i) << endl;
    return 0;
}

```

Contoh 2 — Fibonacci:

```

// Versi rekursif (sederhana tapi lambat untuk n besar)
int fibonacci(int n) {
    if (n == 0) return 0; // base case 1
    if (n == 1) return 1; // base case 2
    return fibonacci(n-1) + fibonacci(n-2); // recursive
}

// Versi iteratif (lebih efisien)
int fibIteratif(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b; a = b; b = c;
    }
    return b;
}

int main() {
    cout << "Fibonacci ke-10: " << fibonacci(10) << endl; // 55
    return 0;
}

```

Contoh 3 — Binary Search Rekursif:

```

int binarySearch(int arr[], int kiri, int kanan, int target) {
    if (kiri > kanan) return -1; // base case: tidak ditemukan
    int tengah = (kiri + kanan) / 2;
    if (arr[tengah] == target) return tengah; // base case: ditemukan
    if (arr[tengah] < target)
        return binarySearch(arr, tengah+1, kanan, target);
    else
        return binarySearch(arr, kiri, tengah-1, target);
}

int main() {
    int data[] = {2, 5, 8, 12, 16, 23, 38, 45};
    int idx = binarySearch(data, 0, 7, 23);
    cout << "23 ditemukan di indeks: " << idx << endl; // 5
    return 0;
}

```

■ *Rekursi dalam (deep recursion) dapat menyebabkan stack overflow. Untuk n besar, pertimbangkan iterasi atau memoization (dynamic programming).*

6. Inline Function

Keyword **inline** meminta compiler untuk menyisipkan kode fungsi langsung di tempat pemanggilan, menghindari overhead pemanggilan fungsi. Cocok untuk fungsi kecil yang dipanggil sangat sering.

```

#include <iostream>
using namespace std;

inline int kuadrat(int x) {
    return x * x;
}

inline double celsiusToFahrenheit(double c) {
    return (c * 9.0 / 5.0) + 32.0;
}

int main() {
    // Compiler mengganti kuadrat(5) dengan (5 * 5) secara langsung
    cout << kuadrat(5) << endl; // 25
    cout << celsiusToFahrenheit(100.0) << endl; // 212
    cout << kuadrat(kuadrat(3)) << endl; // 81
    return 0;
}

```

■ *Inline hanya sebuah saran ke compiler, bukan perintah. Compiler bisa mengabaikannya jika fungsi terlalu kompleks. Compiler modern sering melakukan inlining otomatis.*

7. Lambda Function (C++11)

Lambda adalah fungsi anonim (tanpa nama) yang dapat didefinisikan langsung di tempat penggunaan. Sangat berguna untuk fungsi singkat yang hanya dipakai sekali, terutama saat bekerja dengan algoritma STL.

Sintaks Lambda:

```
[capture](parameter) -> tipe_return { body }
```

// Keterangan capture:

// [] – tidak menangkap variabel dari luar

// [=] – tangkap semua variabel by value

// [&] – tangkap semua variabel by reference

// [x, &y] – x by value, y by reference

Contoh Dasar Lambda:

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    // Lambda sederhana
    auto sapa = [](string nama) {
        cout << "Halo, " << nama << "!" << endl;
    };
    sapa("Budi");

    // Lambda dengan return type
    auto tambah = [](int a, int b) -> int { return a + b; };
    cout << tambah(3, 4) << endl; // 7

    // Lambda dengan capture
    int faktor = 5;
    auto kali = [faktor](int x) { return x * faktor; }; // capture by value
    cout << kali(6) << endl; // 30

    // Lambda dengan capture by reference
    int total = 0;
    auto tambahTotal = [&total](int x) { total += x; };
    tambahTotal(10); tambahTotal(20); tambahTotal(30);
    cout << "Total: " << total << endl; // 60
    return 0;
}
```

Lambda dengan Algoritma STL:

```

#include <algorithm>
#include <vector>
using namespace std;

int main() {
vector<int> angka = {5, 2, 8, 1, 9, 3, 7};

// Sort ascending
sort(angka.begin(), angka.end(), [](int a, int b) { return a < b; });

// Sort descending
sort(angka.begin(), angka.end(), [](int a, int b) { return a > b; });

// Hitung elemen > 5
int jml = count_if(angka.begin(), angka.end(), [](int x) { return x > 5; });
cout << "Elemen > 5: " << jml << endl;

// for_each dengan lambda
for_each(angka.begin(), angka.end(), [](int x) { cout << x << " "; });
return 0;
}

```

8. Function Pointer

Function pointer adalah pointer yang menyimpan alamat sebuah fungsi. Memungkinkan fungsi dipilih dan dipanggil secara dinamis saat runtime — dasar dari callback dan strategi pattern.

```
#include <iostream>
using namespace std;

int tambah(int a, int b) { return a + b; }
int kurang(int a, int b) { return a - b; }
int kali (int a, int b) { return a * b; }

// Tipe: pointer ke fungsi yang menerima (int,int) dan return int
typedef int (*Operasi)(int, int); // atau: using Operasi = int(*)(int,int);

int hitungDengan(int x, int y, Operasi op) {
return op(x, y); // panggil fungsi melalui pointer
}

int main() {
Operasi op = tambah;
cout << op(10, 5) << endl; // 15

op = kurang;
cout << op(10, 5) << endl; // 5

// Array of function pointers
Operasi menu[] = {tambah, kurang, kali};
string nama[] = {"Tambah", "Kurang", "Kali"};

for (int i = 0; i < 3; i++)
cout << nama[i] << "(8,3): " << menu[i](8,3) << endl;

// Kirim sebagai argumen
cout << hitungDengan(20, 4, kali) << endl; // 80
return 0;
}
```

■ Dalam C++ modern, lebih disarankan menggunakan `std::function` dari `<functional>` karena lebih fleksibel dan dapat menampung lambda maupun function pointer sekaligus.

9. Scope & Storage Class

Scope menentukan di mana sebuah variabel bisa diakses. **Storage class** menentukan lifetime dan visibilitas variabel.

Jenis	Deklarasi	Lifetime	Scope
Local	Di dalam fungsi/blok	Selama blok berjalan	Hanya di blok tersebut
Global	Di luar semua fungsi	Selama program	Seluruh file
static local	static int x;	Selama program	Hanya di blok fungsi

Jenis	Deklarasi	Lifetime	Scope
<code>static global</code>	<code>static int x;</code>	Selama program	Hanya file ini (tidak extern)
<code>extern</code>	<code>extern int x;</code>	Bergantung definisi	Antar file .cpp

```
#include <iostream>
using namespace std;

int globalVar = 100; // variabel global

void demoStatic() {
    static int hitungan = 0; // diinisialisasi sekali, nilai bertahan
    hitungan++;
    cout << "Dipanggil ke-" << hitungan << endl;
}

void demoScope() {
    int x = 10; // local variable
    {
        int x = 20; // x berbeda (inner scope)
        cout << "Inner x: " << x << endl; // 20
    }
    cout << "Outer x: " << x << endl; // 10
    cout << "Global : " << globalVar << endl;
}

int main() {
    demoStatic(); // Dipanggil ke-1
    demoStatic(); // Dipanggil ke-2
    demoStatic(); // Dipanggil ke-3
    demoScope();
    return 0;
}
```

■ *Hindari penggunaan variabel global berlebihan. Variabel global menyulitkan debugging dan testing karena bisa diubah dari mana saja dalam program.*

10. Template Function

Template memungkinkan penulisan fungsi generik yang dapat bekerja dengan berbagai tipe data. Compiler akan membuat versi fungsi yang sesuai secara otomatis berdasarkan tipe argumen yang diberikan.

```

#include <iostream>
using namespace std;

// Template function satu tipe
template <typename T>
T maksimum(T a, T b) {
    return (a > b) ? a : b;
}

// Template dengan dua tipe berbeda
template <typename T, typename U>
void cetakPasangan(T pertama, U kedua) {
    cout << pertama << " & " << kedua << endl;
}

// Template dengan non-type parameter
template <typename T, int N>
T jumlahArray(T (&arr)[N]) {
    T total = 0;
    for (int i = 0; i < N; i++) total += arr[i];
    return total;
}

int main() {
    // Compiler generate maksimum<int>, maksimum<double>, maksimum<char>
    cout << maksimum(10, 25) << endl; // 25
    cout << maksimum(3.14, 2.71) << endl; // 3.14
    cout << maksimum('A', 'Z') << endl; // Z

    cetakPasangan("Umur", 20);
    cetakPasangan(3.14, "Pi");

    int angka[] = {1, 2, 3, 4, 5};
    cout << "Jumlah: " << jumlahArray(angka) << endl; // 15
    return 0;
}

```

■ Template adalah dasar dari STL (Standard Template Library) seperti vector, sort, dan find. Template dievaluasi saat compile time, sehingga tidak ada overhead runtime.

11. Ringkasan & Best Practices

Konsep	Kapan Digunakan
Fungsi biasa	Logika yang digunakan berulang kali
void function	Fungsi yang hanya melakukan aksi tanpa nilai kembali
By reference param	Ubah nilai asli atau hindari copy data besar

Konsep	Kapan Digunakan
Const reference param	Data besar yang hanya dibaca (efisien)
Default parameter	Parameter opsional dengan nilai standar
Function overloading	Operasi serupa pada tipe berbeda
Rekursi	Masalah yang dapat dipecah menjadi submasalah serupa
Inline function	Fungsi sangat kecil yang dipanggil sangat sering
Lambda	Fungsi singkat sekali pakai, terutama dengan STL
Function pointer	Pemilihan fungsi secara dinamis (callback)
Template function	Algoritma generik untuk berbagai tipe data

Tips Best Practice:

- Satu fungsi sebaiknya melakukan **satu tugas** saja (*Single Responsibility*).
- Panjang fungsi idealnya tidak lebih dari **20-30 baris** agar mudah dibaca.
- Gunakan **nama fungsi yang deskriptif**: cetakLaporan() lebih baik dari f1().
- Pilih **const reference** untuk parameter objek/string besar yang tidak diubah.
- Selalu pastikan fungsi rekursif memiliki **base case** yang benar.
- Gunakan **prototype (deklarasi)** di header file untuk fungsi yang dipakai di banyak file.

Selamat belajar C++ lebih lanjut!

Modul berikutnya: OOP — Class, Inheritance & Polymorphism